

Sesión 9: Responsividad Avanzada, Adaptive UI con LayoutBuilder, MediaQuery y Breakpoints

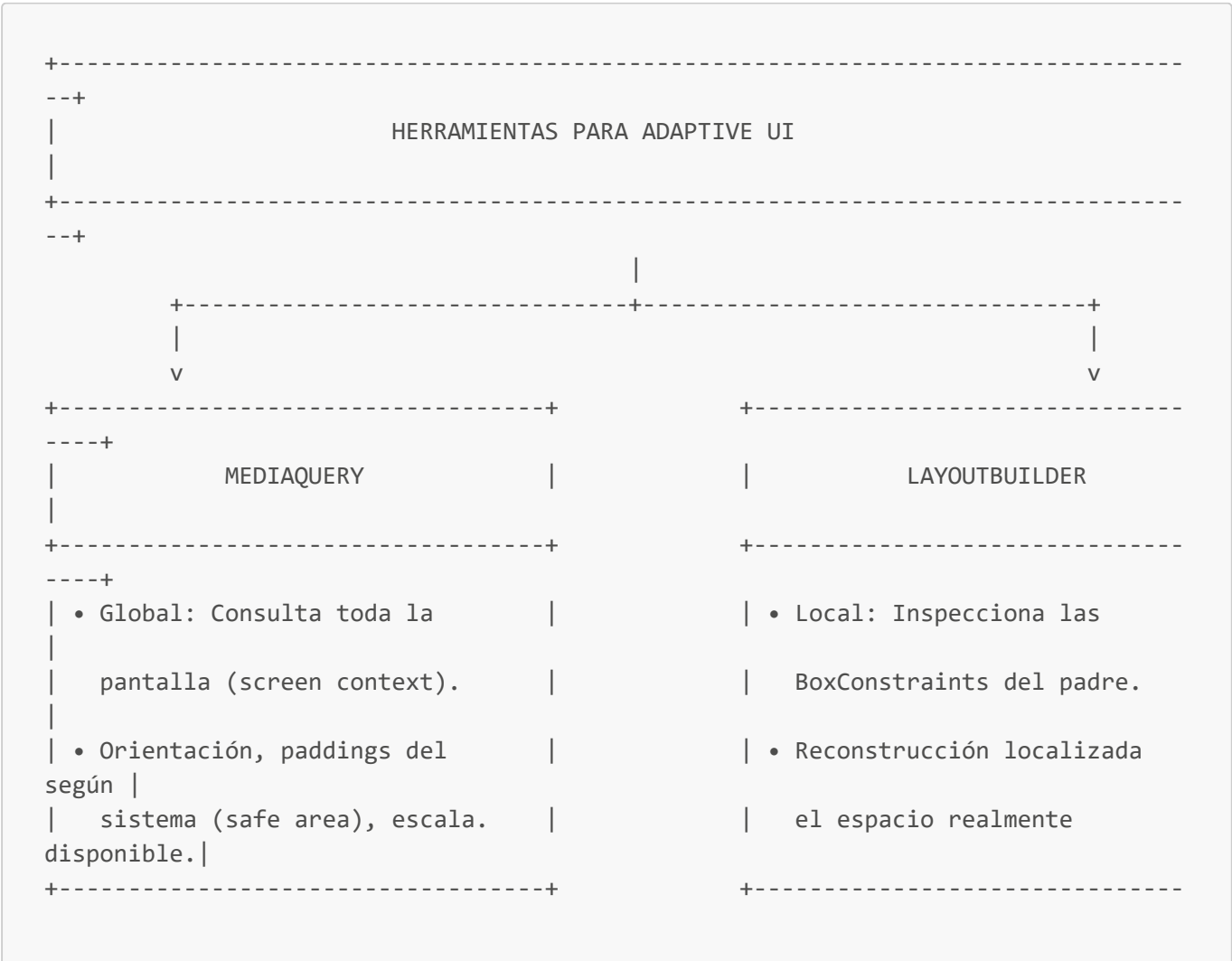
Introducción

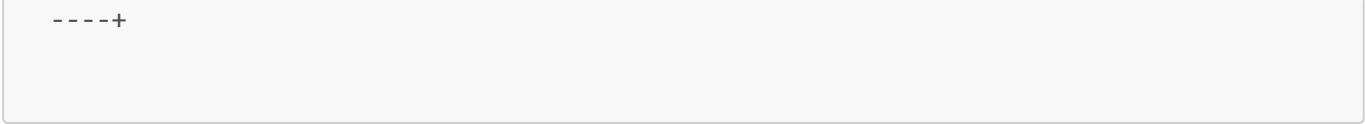
Tras haber analizado el modelo profundo de restricciones geométricas (**BoxConstraints**) en la sesión anterior, resulta esencial trasladar estos conceptos hacia el diseño de interfaces adaptativas (*Adaptive UI*). El desarrollo multiplataforma moderno exige que las aplicaciones respondan dinámicamente no solo a la orientación del dispositivo, sino a una amplia gama de dimensiones de pantalla, desde dispositivos móviles compactos hasta monitores de escritorio de alta resolución.

En esta sesión profundizaremos en el diseño adaptativo en Flutter mediante el uso combinado de **MediaQuery** y **LayoutBuilder**. Aprenderemos a definir y utilizar puntos de interrupción (*breakpoints*) estandarizados para reorganizar estructuras de maquetación complejos y ofrecer experiencias de usuario óptimas en cualquier factor de forma.

Conceptos y Definiciones

El diseño de interfaces adaptativas en Flutter se apoya en dos herramientas complementarias que inspeccionan el entorno espacial a diferentes niveles de la jerarquía de widgets.





1. MediaQuery vs LayoutBuilder

- **MediaQuery:**
 - Obtiene información **global** sobre el contexto de la pantalla donde se ejecuta la aplicación (`MediaQuery.of(context)`).
 - Devuelve parámetros como `size.width`, `size.height`, `orientation`, `devicePixelRatio`, y los márgenes seguros (`padding` para muescas/islas dinámicas).
 - **Consideración de Rendimiento:** Usar `MediaQuery.of(context).size` provoca que el widget se vuelva a renderizar cuando **cualquier** propiedad del `MediaQueryData` cambie (por ejemplo, cuando aparece el teclado virtual). Para optimizar, se prefiere la sintaxis fina como `MediaQuery.sizeOf(context)`.
- **LayoutBuilder:**
 - Evalúa las restricciones geométricas **locales** (`BoxConstraints`) enviadas por el widget padre directo.
 - Permite construir layouts adaptativos para componentes específicos (como tarjetas, paneles o diálogos) sin importar la dimensión total de la pantalla.
 - Facilita la modularización de componentes que deben reorganizarse según el tamaño de su contenedor contenedor.

2. Estructuración de Breakpoints

Los puntos de interrupción (*breakpoints*) son umbrales de ancho de pantalla definidos para transicionar de un esquema de diseño a otro (por ejemplo, pasar de una vista de columna única en móviles a un diseño con panel lateral en tablets y escritorio).

Alineado con las guías de **Material Design 3**, los rangos estándar sugeridos se dividen en:

Categoría	Breakpoint (Ancho en dp)	Dispositivos Típicos	Estrategia de Layout
Compact	< 600 dp	Smartphones en retrato	Layout vertical único, <code>BottomNavigationBar</code> , navegación por pila.
Medium	600 dp a 839 dp	Tablets pequeñas, plegables, smartphones en paisaje	Layout de dos columnas, <code>NavigationRail</code> lateral.
Expanded	≥ 840 dp	Tablets grandes, monitores de escritorio y Web	Layout multicolumna, <code>NavigationDrawer</code> permanente o paneles máster-detalle.

3. Estrategias de Arquitectura Adaptativa

1. **Reorganización de Vistas (Reflow):** Cambiar arreglos de un solo eje (**Column**) a múltiples ejes (**Row** o **GridView**).
2. **Exposición Gradual de Contenido (Show/Hide):** Mostrar información secundaria o paneles de filtros únicamente cuando el ancho supera los 840 dp.
3. **Sustitución de Navegación:** Intercambiar de forma transparente la barra de navegación inferior (**BottomNavigationBar**) por un riel de navegación vertical (**NavigationRail**) al pasar de *Compact* a *Medium/Expanded*.

Ejemplo Completo con Código Fuente: Dashboard Adaptativo Multi-Breakpoints

El siguiente ejemplo implementa una interfaz responsiva completa que inspecciona las dimensiones locales y globales mediante **LayoutBuilder** y **MediaQuery.sizeOf**, ajustando la estructura de navegación y el tablero de contenido dinámicamente según los *breakpoints* establecidos.

Código Fuente Explicado (**lib/main.dart**)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoAdaptiveUIApp());
}

/// Widget Raíz
class DemoAdaptiveUIApp extends StatelessWidget {
  const DemoAdaptiveUIApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Adaptive UI',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaDashboardAdaptativo(),
    );
  }
}

/// Pantalla Principal Adaptativa
class PantallaDashboardAdaptativo extends StatefulWidget {
  const PantallaDashboardAdaptativo({super.key});

  @override
```

```

State<PantallaDashboardAdaptativo> createState() =>
  _PantallaDashboardAdaptativoState();
}

class _PantallaDashboardAdaptativoState
  extends State<PantallaDashboardAdaptativo> {
  int _indiceSeleccionado = 0;

  @override
  Widget build(BuildContext context) {
    // Uso optimizado de MediaQuery para obtener el ancho total de la pantalla
    final double anchoPantalla = MediaQuery.sizeOf(context).width;

    // Determinación de Breakpoints
    final bool esCompacto = anchoPantalla < 600;
    final bool esMedio = anchoPantalla >= 600 && anchoPantalla < 840;

    return Scaffold(
      appBar: AppBar(
        title: Text(
          esCompacto
            ? 'Dashboard (Móvil)'
            : (esMedio ? 'Dashboard (Tablet)' : 'Dashboard (Escritorio)'),
        ),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),

      // ESTRATEGIA DE NAVEGACIÓN ADAPTATIVA
      // En pantalla compacta usaremos BottomNavigationBar, en pantallas
      // medianas/grandes usaremos NavigationRail
      body: Row(
        children: [
          if (!esCompacto)
            NavigationRail(
              selectedIndex: _indiceSeleccionado,
              onDestinationSelected: (int index) {
                setState(() => _indiceSeleccionado = index);
              },
              labelType: esMedio
                ? NavigationRailLabelType.selected
                : NavigationRailLabelType.all,
              destinations: const [
                NavigationRailDestination(
                  icon: Icon(Icons.dashboard_outlined),
                  selectedIcon: Icon(Icons.dashboard),
                  label: Text('Tablero'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.analytics_outlined),
                  selectedIcon: Icon(Icons.analytics),
                  label: Text('Métricas'),
                ),
                NavigationRailDestination(

```

```

        icon: Icon(Icons.settings_outlined),
        selectedIcon: Icon(Icons.settings),
        label: Text('Ajustes'),
      ),
    ],
  ),
  if (!esCompacto) const VerticalDivider(thickness: 1, width: 1),

  // CONTENIDO PRINCIPAL REORGANIZABLE CON LAYOUTBUILDER
  Expanded(
    child: LayoutBuilder(
      builder: (context, constraints) {
        return ContenidoTableroAdaptativo(
          anchoDisponible: constraints.maxWidth,
        );
      },
    ),
  ),
],
),

bottomNavigationBar: esCompacto
? BottomNavigationBar(
  currentIndex: _indiceSeleccionado,
  onTap: (index) => setState(() => _indiceSeleccionado = index),
  items: const [
    BottomNavigationBarItem(
      icon: Icon(Icons.dashboard),
      label: 'Tablero',
    ),
    BottomNavigationBarItem(
      icon: Icon(Icons.analytics),
      label: 'Métricas',
    ),
    BottomNavigationBarItem(
      icon: Icon(Icons.settings),
      label: 'Ajustes',
    ),
  ],
)
: null,
);
}
}

/// Componente que reorganiza las tarjetas según el ancho que le asigna el padre
class ContenidoTableroAdaptativo extends StatelessWidget {
  final double anchoDisponible;

  const ContenidoTableroAdaptativo({
    super.key,
    required this.anchoDisponible,
  });
}

```

```
@override
Widget build(BuildContext context) {
  // Definición de columnas del Grid según el ancho local provisto por
  LayoutBuilder
  int columnasGrid = 1;
  if (anchoDisponible >= 840) {
    columnasGrid = 3;
  } else if (anchoDisponible >= 500) {
    columnasGrid = 2;
  }

  return SingleChildScrollView(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          'Espacio local disponible: ${anchoDisponible.toStringAsFixed(1)} dp',
          style: const TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 16),

        // Reorganización en rejilla adaptativa
        GridView.builder(
          shrinkWrap: true,
          physics: const NeverScrollableScrollPhysics(),
          itemCount: 6,
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: columnasGrid,
            crossAxisSpacing: 12,
            mainAxisSpacing: 12,
            childAspectRatio: 1.6,
          ),
          itemBuilder: (context, index) {
            return Card(
              elevation: 2,
              color: Theme.of(context).colorScheme.surfaceContainerHigh,
              child: Padding(
                padding: const EdgeInsets.all(16.0),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(
                      Icons.memory,
                      color: Theme.of(context).colorScheme.primary,
                      size: 32,
                    ),
                    const SizedBox(height: 8),
                    Text(
                      'Módulo #${index + 1}',
                      style: const TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,

```

```
    ),  
    ),  
    const Text(  
      'Estado del sistema: Normal',  
      style: TextStyle(fontSize: 12, color: Colors.grey),  
    ),  
  ],  
),  
),  
);  
},  
,  
),  
],  
,  
);  
}  
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de un Panel Máster-Detalle Adaptativo

1. Crea un layout de dos paneles para mostrar una lista de elementos (Panel Máster) y la vista detallada de un elemento seleccionado (Panel Detalle).
2. Usa `LayoutBuilder` para evaluar el ancho local disponible:
 - Si `maxWidth < 700 dp`: Implementa navegación jerárquica (al tocar un elemento se navega mediante `Navigator.push` a la pantalla de detalle).
 - Si `maxWidth >= 700 dp`: Renderiza en un mismo `Row` ambos paneles lado a lado (`Expanded(flex: 1, child: ListaMaster)` y `Expanded(flex: 2, child: VistaDetalle)`).

Práctica 2: Gestión de Orientación y Márgenes Seguros (`SafeArea`)

1. Utiliza `MediaQuery.orientationOf(context)` para detectar si el usuario rotó la pantalla a modo Horizontal (`Landscape`).
2. Inspecciona los paddings generados por la muesca del sistema usando `MediaQuery.paddingOf(context)`.
3. Integra el widget `SafeArea` para garantizar que los elementos de texto e íconos no queden ocultos debajo de la cámara frontal ni de los bordes redondeados del dispositivo.

Resumen y Puntos Clave

- **Diseño Adaptativo vs Responsivo:** El diseño responsivo ajusta proporciones de elementos; el diseño adaptativo modifica activamente la estructura y componentes de la UI (ej. intercambiando `BottomNavigationBar` por `NavigationRail`).

- **MediaQuery vs LayoutBuilder:** **MediaQuery** provee el contexto global de la pantalla. **LayoutBuilder** inspecciona las **BoxConstraints** locales del contenedor padre, facilitando componentes desacoplados y reusables.
 - **Optimización de MediaQuery:** Es recomendable utilizar llamadas directas especializadas como **MediaQuery.sizeOf(context)** u **orientationOf(context)** para evitar reconstrucciones innecesarias ante cambios no relacionados.
 - **Breakpoints Estándar (Material 3):**
 - **Compact:** < 600 dp (Móvil).
 - **Medium:** 600 a 839 dp (Tablet/Plegable).
 - **Expanded:** ≥ 840 dp (Escritorio/Web).
-

Bibliografía y Referencias

1. **Google LLC.** (2024). *Creating responsive and adaptive apps in Flutter*. Recuperado de: <https://docs.flutter.dev/ui/layout/responsive-adaptive>
2. **Google LLC.** (2024). *Material Design 3 - Layout Breakpoints*. Recuperado de: <https://m3.material.io/foundations/layout/applying-layout/window-size-classes>
3. **Google LLC.** (2024). *LayoutBuilder class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/widgets/LayoutBuilder-class.html>
4. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.